

CivicLens

Setup & Deployment Guide

City-Scale Video Intelligence Platform

SQLite / PostgreSQL / MySQL

Redis Message Queue

Edge AI Mode

Docker Deployment

Multi-Server Architecture

Security Hardening

Version 1.0 | September 2026

Usman Farid | 0317 750 5992 | Made in Pakistan

CivicLens - Complete Setup & Deployment Guide

Last Updated: 2026-09-01 | Version: 1.0

Table of Contents

- 4. Message Queue (Redis)
- 5. Edge AI Mode
- 6. Multi-Server / Distributed Setup
- 7. Docker Deployment
- 8. Security Hardening
- 9. Configuration Reference
- 10. Troubleshooting

1. Quick Start (Single Machine)

Use case: Building, campus, small facility - up to 50 cameras

Database: SQLite (zero setup)

Time: 5 minutes

```
Double-click: CivicLens-Setup.exe
?
UAC Prompt -> Click "Yes" (admin permission)
?
Install Wizard -> Choose install location (default: C:\Program Files\CivicLens)
?
Click "Install"
?
Check "Launch CivicLens Command Center" -> Click "Finish"
```

```
????????????????????????????????????????????????????????
?  CivicLens Command Center                                ?
?                                                         ?
?  First-run bootstrap                                    ?
?  Create an administrator username and password          ?
?                                                         ?
?  [Create Administrator]                                  ?
????????????????????????????????????????????????????????
```

```
Sidebar -> Cameras -> "Add Camera"
?
Name: Gate 1
RTSP URL: rtsp://192.168.1.100:554/stream1
?
Click "Save" -> Camera appears on Camera Wall
```

2. PostgreSQL Setup (City-Scale)

Use case: City-wide deployment, 100-500+ cameras, multi-server

Time: 20 minutes

1. Download: <https://www.postgresql.org/download/windows/>
2. Run installer
3. Set password for 'postgres' user (remember it!)
4. Port: 5432 (default)
5. Finish installation

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

Open pgAdmin (comes with PostgreSQL) or use command line:

```
# Command line method:
psql -U postgres

postgres=# CREATE DATABASE civiclens;
postgres=# CREATE USER civiclens_user WITH PASSWORD 'your_strong_password';
postgres=# GRANT ALL PRIVILEGES ON DATABASE civiclens TO civiclens_user;
postgres=# \c civiclens
postgres=# GRANT ALL ON SCHEMA public TO civiclens_user;
postgres=# \q
```

```
CREATE DATABASE
CREATE ROLE
GRANT
GRANT
```

Navigate to your CivicLens installation folder:

```
C:\Program Files\CivicLens\backend\
```

Copy `.env.example` to `.env`:

```
copy .env.example .env
```

Edit `.env`:

```
# Change this line:
DATABASE_URL=postgresql://civiclens_user:your_strong_password@localhost:5432/civiclens
```

```
# In the CivicLens backend folder:
cd "C:\Program Files\CivicLens\backend\venv\Scripts"
.\pip.exe install psycopg2-binary
```

1. Close CivicLens (Ctrl+C in console or close the window)
2. Double-click CivicLens.exe again
3. Tables auto-created in PostgreSQL
4. Login and verify

```
psql -U civiclens_user -d civiclens -c "\dt"
```

List of relations

Schema	Name	Type	Owner
public	alembic_version	table	civiclens_user
public	alerts	table	civiclens_user
public	cameras	table	civiclens_user
public	civic_events	table	civiclens_user
public	detections	table	civiclens_user
public	missing_persons	table	civiclens_user
public	participants	table	civiclens_user
public	users	table	civiclens_user
public	watchlist	table	civiclens_user
... (17 tables total)			

3. MySQL Setup (Alternative)

1. Download: <https://dev.mysql.com/downloads/installer/>
2. Run MySQL Installer
3. Choose "Server only"
4. Set root password
5. Finish

```
mysql -u root -p

mysql> CREATE DATABASE civiclens CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
mysql> CREATE USER 'civiclens_user'@'localhost' IDENTIFIED BY 'your_password';
mysql> GRANT ALL PRIVILEGES ON civiclens.* TO 'civiclens_user'@'localhost';
mysql> FLUSH PRIVILEGES;
mysql> EXIT;
```

```
DATABASE_URL=mysql+pymysql://civiclens_user:your_password@localhost:3306/civiclens
```

```
pip install pymysql cryptography
```

4. Message Queue (Redis)

Use case: 100+ cameras, distributed processing across multiple servers

Time: 15 minutes

1. Download: <https://github.com/tporadowski/redis/releases>
2. Extract to C:\Redis
3. Run: C:\Redis\redis-server.exe
4. Default port: 6379

```
sudo apt install redis-server
sudo systemctl start redis
sudo systemctl enable redis
```

```
docker run -d --name redis -p 6379:6379 redis:7-alpine
```

Edit `backend/.env`:

```
MESSAGE_QUEUE=redis://localhost:6379
QUEUE_CHANNEL=civiclens:detections
```

```
Close CivicLens -> Double-click CivicLens.exe
Backend log will show: "Message queue connected: redis://localhost:6379"
```

```

?????????????   publish   ??????????   consume   ??????????????
? Camera  ? ??????????????????-> ? Redis ? ??????????????????-> ? Worker  ?
? Detection?           ? Queue ?           ? Node      ?
????????????????           ??????????           ??????????????
?                               ?
???????????????????????? results/alerts <-????????????????????????
```

- Detection results are published to Redis instead of processed inline
- Worker nodes consume events and process them asynchronously
- Scales horizontally - add more workers to handle more cameras

5. Edge AI Mode

Use case: Resource-constrained devices, on-camera processing, Raspberry Pi, low-end servers

Time: 2 minutes (just edit .env)

Edit `backend/.env`:

```
EDGE_MODE=1
```

```
EDGE_IMG_SIZE=320      # Lower resolution = faster (default: 320)
EDGE_FRAME_SKIP=3      # Process every 3rd frame (default: 3)
EDGE_CONF_THRESHOLD=0.35 # Higher = fewer detections, less load
EDGE_SKIP_SCENE=1      # Skip heavy scene analysis (SegFormer)
```

Setting	Full Mode	Edge Mode
----- ----- -----		
Model	yolov8n (640px)	yolov8n (320px)
Frame Skip	None (every frame)	Every 3rd frame
Confidence	0.25	0.35
Scene Analysis	Full	Skipped
GPU Required	Recommended	Not needed (CPU OK)
RAM Usage	~2GB	~500MB

No code changes needed - just restart CivicLens.exe.

6. Multi-Server / Distributed Setup

Use case: 500+ cameras across multiple locations

```

????????????????      ?????????????????      ?????????????????
?  Edge Node 1  ?      ?  Edge Node 2  ?      ?  Edge Node 3  ?
?  (50 cameras)?      ?  (50 cameras)?      ?  (50 cameras)?
?  GPU Server  ?      ?  GPU Server  ?      ?  GPU Server  ?
????????????????      ?????????????????      ?????????????????
      ?                  ?                  ?
      ?????????????????????????????????????????????????????
              ?
              ?????????????
              ? PostgreSQL ?
              ? (Central) ?
              ?????????????
              ?
              ?????????????
              ? Frontend  ?
              ? (Next.js) ?
              ?????????????

```

Install PostgreSQL on a dedicated server:

```

Server: db.civiclens.local
Port: 5432
Storage: 500GB SSD minimum
RAM: 16GB minimum

```

Each edge node runs CivicLens backend only (no frontend):

```

# On each edge node:
1. Install CivicLens
2. Edit backend/.env:
  DATABASE_URL=postgresql://civiclens_user:password@db.civiclens.local:5432/civiclens

3. Start only backend:
  cd backend
  .\env\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000

```

Run Next.js frontend on a separate server:

```
# Frontend server:
1. Install Node.js 20+
2. cd civiclens-frontend
3. Edit .env.local:
    NEXT_PUBLIC_API_URL=http://edge-node-1.local:8000

4. npm run build
5. node server.js
```

Use Nginx to distribute API requests across edge nodes:

```
upstream civiclens_backends {
    server edge-node-1.local:8000;
    server edge-node-2.local:8000;
    server edge-node-3.local:8000;
}

server {
    listen 80;
    location /api/ {
        proxy_pass http://civiclens_backends;
    }
}
```

7. Docker Deployment

```
version: "3.8"

services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: civiclens
      POSTGRES_USER: civiclens_user
      POSTGRES_PASSWORD: your_password
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://civiclens_user:your_password@db:5432/civiclens
    ports:
      - "8000:8000"
    depends_on:
      - db
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]

  frontend:
    build: ./civiclens-frontend
    environment:
      NEXT_PUBLIC_API_URL: http://backend:8000
    ports:
      - "3000:3000"
    depends_on:
      - backend

volumes:
  pgdata:
```

```
docker-compose up -d --build
```

8. Security Hardening

Create a strong administrator password during first-run bootstrap. To change it later:

```
Settings -> Change Password  
Current: [Your current password]  
New: [A unique strong password]
```

```
? Use VPN for remote access  
? Firewall: Only allow ports 3000, 8000 from trusted IPs  
? HTTPS: Use Nginx + Let's Encrypt for SSL  
? Change SECRET_KEY in .env to a random string
```

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Output:

```
dGhpcy1pcy1hLXNlY3JldC1rZXktY2hhbmdlLW1lLXBsZWZlZQ
```

Add to `.env`:

```
SECRET_KEY=dGhpcy1pcy1hLXNlY3JldC1rZXktY2hhbmdlLW1lLXBsZWZlZQ
```

9. Configuration Reference

All settings are configured via ``backend/.env``. Copy ``env.example`` to ``env`` to get started.

Variable	Default	Description
----- ----- -----		
<code>`DATABASE_URL`</code>	<code>`sqlite:///civiclens.db`</code>	Database connection string
<code>`MESSAGE_QUEUE`</code>	<code>`_(empty)_`</code>	Redis URL for message queue
<code>`QUEUE_CHANNEL`</code>	<code>`civiclens:detections`</code>	Redis pub/sub channel
<code>`EDGE_MODE`</code>	<code>`0`</code>	Enable lightweight edge processing
<code>`EDGE_IMG_SIZE`</code>	<code>`640` / <code>`320`</code></code>	Inference resolution
<code>`EDGE_FRAME_SKIP`</code>	<code>`1` / <code>`3`</code></code>	Process every Nth frame
<code>`EDGE_CONF_THRESHOLD`</code>	<code>`0.25` / <code>`0.35`</code></code>	Detection confidence
<code>`EDGE_SKIP_SCENE`</code>	<code>`0` / <code>`1`</code></code>	Skip SegFormer scene analysis
<code>`WORKER_ONLY`</code>	<code>`0`</code>	Backend-only mode (no frontend)
<code>`STORAGE_RETENTION_DAYS`</code>	<code>`30`</code>	Auto-delete old images
<code>`BACKEND_PORT`</code>	<code>`8000`</code>	Backend API port
<code>`FRONTEND_PORT`</code>	<code>`3000`</code>	Frontend UI port
<code>`SECRET_KEY`</code>	<code>`_(built-in)_`</code>	JWT signing key (change in production)

```
# No .env needed - defaults work perfectly
```

```
DATABASE_URL=postgresql://user:pass@db-server:5432/civiclens
MESSAGE_QUEUE=redis://redis-server:6379
STORAGE_RETENTION_DAYS=60
```

```
DATABASE_URL=postgresql://user:pass@central-db:5432/civiclens
WORKER_ONLY=1
EDGE_MODE=1
EDGE_FRAME_SKIP=5
```

10. Troubleshooting

Cause: SQLite can't handle concurrent writes from many cameras

Fix: Switch to PostgreSQL (see Section 2)

```
# Check PostgreSQL is running:
sudo systemctl status postgresql

# Check pg_hba.conf allows connections:
# Add this line to pg_hba.conf:
host  civicLens  civicLens_user  192.168.1.0/24  md5

# Restart PostgreSQL:
sudo systemctl restart postgresql
```

```
cd "C:\Program Files\CivicLens\backend\venv\Scripts"
.\pip.exe install psycpg2-binary
```

1. Check RTSP URL is correct
2. Verify camera is reachable: ping 192.168.1.100
3. Check port 554 is open on camera
4. Try VLC: Media -> Open Network Stream -> paste RTSP URL

Solutions:

1. Reduce FPS in camera settings (default: 1 FPS for detection)
2. Use smaller YOLO model (yolov8n instead of yolov8s)
3. Add frame skipping (process every 5th frame)
4. Add more edge inference nodes

Quick Reference

Scenario	Database	Config
----- ----- -----		
Single building (?50 cams)	SQLite	Default - no setup
City district (50-200 cams)	PostgreSQL	Edit <code>.env`</code>
Full city (200+ cams)	PostgreSQL + Edge nodes	Multi-server
Enterprise (500+ cams)	PostgreSQL cluster + K8s	Docker + orchestration

Support

Developer: Usman Farid

Phone: 0317 750 5992

Made in: Pakistan

This document is part of CivicLens Command Center v1.0